
CS 61B

Spring 2025

Midterm 2

Wednesday, April 2, 2025

Your Name: _____

Your SID: _____ Location: _____

SID of person to your left: _____ Right: _____

This exam is worth **100 points**, and consists of 6 questions. Questions are ordered by topic, not by difficulty.

Question:	1	2	3	4	5	6	Total
Points:	16	14	31	16	12	11	100

- ☐ indicates only one circle should be filled in. ☐ indicates more than one box may be filled in. **Please fill in the shape completely.** If you change your response, **erase as completely as possible.**
- Anything you write that you ~~cross out~~ will not be graded.
- If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade according to the worst interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) is relevant, it will be explicitly noted in the question.

Coding questions:

- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B style guidelines. For example, any if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported.

Write the statement below in the same handwriting you will use on the rest of the exam. Failure to do so may result in a voided exam.

I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat. If these answers are not my own work, I may be deducted up to 9,876,543,210 points.

Signature: _____

1 Potpourri

(16 points)

- (a) (1 point) True/False: All valid Left-Leaning Red-Black Trees have the same number of black links along every root-to-leaf path.

☐ True

☐ False

- (b) (1 point) True/False: If a binary search tree has its edges colored red/black such that the number of black links along every root-to-leaf path is the same, then it is a valid Left-Leaning Red-Black Tree.

☐ True

☐ False

- (c) (1 point) True/False: If all edges in an undirected, connected graph have the same weight, any spanning tree of the graph is a valid Minimum Spanning Tree.

☐ True

☐ False

- (d) (1 point) True/False: While running Kruskal's algorithm, we multiply all edge weights of an undirected, connected graph by -1 before inserting them into the heap. The resulting spanning tree will be a **Maximum** Spanning Tree of the original graph.

☐ True

☐ False

- (e) (1 point) True/False: Let G be an undirected, connected graph with a unique Minimum Spanning Tree, and let C be a cut of G with more than one crossing edge. The edge with the maximum weight across C is never included in the resulting Minimum Spanning Tree.

☐ True

☐ False

- (f) (1 point) True/False: In Quick Find, the isConnected operation has a runtime of $O(1)$.

☐ True

☐ False

- (g) (1 point) What is the fewest number of edges in an undirected graph with N vertices and exactly 3 connected components?

☐ 1 edge

☐ $N - 1$ edges

☐ 2 edges

☐ $N - 2$ edges

☐ 3 edges

☐ $N - 3$ edges

☐ 4 edges

☐ None of the above

For parts h and i, consider a modified heap, where instead of using a resizing array to store elements, we use a circular doubly-linked list with a sentinel.

- (h) (2 points) What is the best-case runtime for `removeMin` in this heap, assuming the heap contains N elements?

- | | |
|---|---|
| ☐ $\Theta(1)$ | ☐ $\Theta(N \log(N))$ |
| ☐ $\Theta(\log(N))$ | ☐ $\Theta(N^2)$ |
| ☐ $\Theta(N)$ | ☐ None of the Above |

- (i) (2 points) What is the best-case runtime for `insert` in this heap, assuming the heap contains N elements?

- | | |
|---|---|
| ☐ $\Theta(1)$ | ☐ $\Theta(N \log(N))$ |
| ☐ $\Theta(\log(N))$ | ☐ $\Theta(N^2)$ |
| ☐ $\Theta(N)$ | ☐ None of the Above |

- (j) (2 points) Which of the following is true about the array, `[-, 0, 0, 2, 5, 0, 7, 5]`? Select all that apply.

- ☐ The array is a valid backing array of a max-heap.
- ☐ The array is a valid backing array of a min-heap.
- ☐ If the `-` is replaced with `-8`, the array is a valid backing array of a Weighted Quick Union **without** path compression.
- ☐ If the `-` is replaced with `-8`, the array is a valid backing array of a Weighted Quick Union **with** path compression.
- ☐ None of the above

- (k) (2 points) Suppose a binary search tree contains the following elements: `{1, 2, 3, 4, 5, 6, 7, 8}`, with the root containing 7. After deleting the root node using Hibbard deletion, which of the following nodes could possibly be the new root of the tree? Select all that apply.

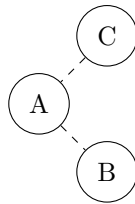
- | | |
|----------------------------|---|
| ☐ 1 | ☐ 5 |
| ☐ 2 | ☐ 6 |
| ☐ 3 | ☐ 8 |
| ☐ 4 | ☐ None of the Above |

- (l) (1 point) Given the backing array `parents` of a Weighted Quick Union, which of these returns the size of the largest connected component?

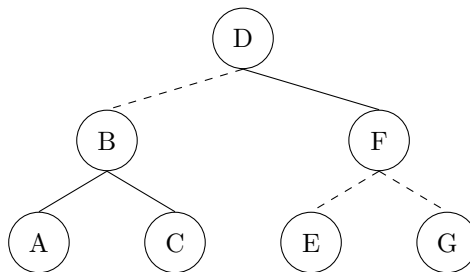
- ☐ `min(parents)`
- ☐ `max(parents)`
- ☐ `max(parents) * -1`
- ☐ `min(parents) * -1`
- ☐ None of the above

The trees below represent LLRBs immediately after a new element has been added, but before any invariant-fixing operations have been taken (dotted edges represent red edges and solid edges represent black edges). For each tree, write out the sequence of operations (ex. `rotateLeft(A)`, `rotateRight(B)`, `colorFlip(C)`) needed to fix all invariants. If it is impossible to have created the LLRB by adding a single new element, write “Not possible”. Note that the letters are simply placeholders.

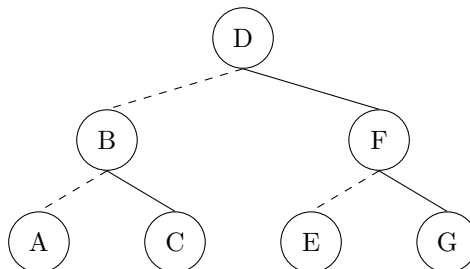
(b) (3 points)



(c) (3 points)



(d) (3 points)



3 Anirudh's Weighted Question

(31 points)

Anirudh the Adventurer wants to find a treasure hidden on one of N islands. Luckily, he has discovered the secret library of Stella the Sneaky Sorceress. The library contains many scrolls that each represent a map of the N islands. On each scroll, there is an array of N numbers. Each array is a representation of a Quick Union tree with a single root. For example, for $N = 5$, one possible array might be $[1, 2, 3, -5, 0]$.

Anirudh knows that all of the Quick Union trees are valid, but only one of them is a valid Weighted Quick Union (without path compression). The one scroll with a valid Weighted Quick Union is the only real map. By “valid WQU”, we mean that the given array can be created by a sequence of union operations (under some tie-breaking scheme). For example, $[1, 2, 3, -5, 0]$ is a valid QU, but not a valid WQU.

To determine which map is real, he will first convert each array into a `TreasureMap`, which is a linked data structure rather than an array. Note: Anirudh's nested `Island` class represents a node of the tree, and each node keeps track of its children. Then he will call `isValid` on the `TreasureMap`. Fill in the functions in the `TreasureMap` class to help Anirudh find the treasure.

```
public class TreasureMap {

    private Island root;

    private class Island {
        public int label;
        public List<Island> children;

        public Island(int label) {
            this.label = label;
            this.children = new ArrayList<>();
        }
    }

    // Converts the underlying array representation of a QuickUnion into a corresponding TreasureMap.
    public TreasureMap(int[] qu) {
        // part a
    }

    // Checks if the TreasureMap is a valid WeightedQuickUnion
    public boolean isValid() {
        // part e
    }

    // Helper function for isValid
    // Returns -1 if parent is not a valid WQU
    // Otherwise returns the number of nodes in the subtree rooted at parent
    private int size(Island parent) {
        // part e
    }
}
```

- (a) (8 points) Complete the constructor of the `TreasureMap` class, which receives as input the backing array of a Quick Union with exactly one root.

```
public TreasureMap(int[] qu) {
    Island[] isles = new Island[qu.length];
    for (int i = 0; i < qu.length; i++) {
        isles[i] = new Island(i);
    }

    for (_____1_____; _____2_____; _____3_____) {
        int parent = qu[i];

        if (_____4_____) {
            _____5_____;
        } else {
            _____6_____;
        }
    }
}
```

- (b) (4 points) What are the best-case and worst-case runtimes of this function? Express your answer as a Θ bound in terms of N , the number of islands.

Best-Case Runtime:

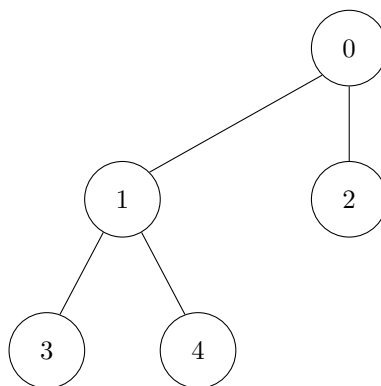
 $\Theta(\quad)$

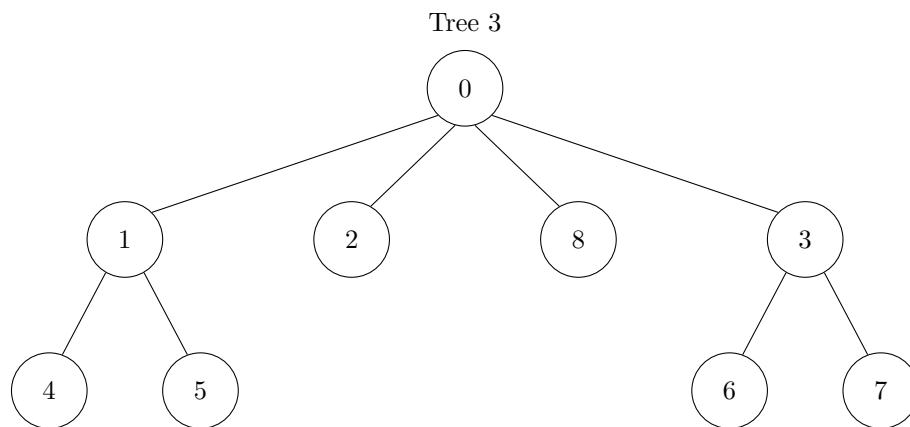
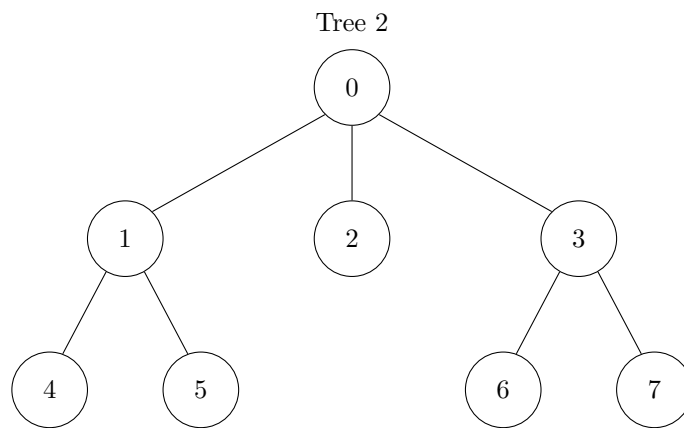
Worst-Case Runtime:

 $\Theta(\quad)$

- (c) (2 points) In order to gain some intuition behind the problem, Anirudh decides to consider three small test cases to see what properties valid Weighted Quick Unions (WQUs) have. **Two of the trees below are invalid, and one is valid.** Determine which tree is valid. The next two trees are on the following page.

Tree 1

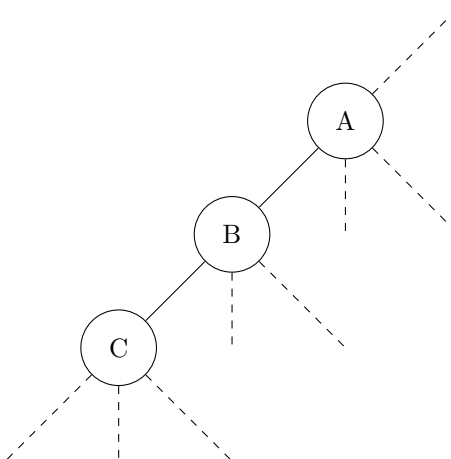



☐ Tree 1 is valid

☐ Tree 2 is valid

☐ Tree 3 is valid

- (d) (1 point) True/False: Consider a WQU where node A is a parent of node B, and node B is a parent of node C. Node C must have been connected to node B before node B was connected to node A.


☐ True

☐ False

- (e) (16 points) Complete the `isValid` method, which determines if the `TreasureMap` corresponds to a valid Weighted Quick Union, and the `size` helper method. Your solution must run in $O(N \log N)$ time, where N is the number of islands.

Warning: We consider this question to be significantly harder than the other questions on the exam.

// Checks if the `TreasureMap` is a valid `WeightedQuickUnion`

public boolean isValid() {

_____ 1;
}

// Helper function for `isValid`

// Returns -1 if parent is not a valid WQU

// Otherwise returns the number of nodes in the subtree rooted at parent

private int size(Island parent) {

int currSize = 1;

 _____ 2 childSizes = **new** _____ 3;

for (_____ 4 : _____ 5) {

 _____ 6;
 }

while (_____ 7) {

 _____ 8;

if (_____ 9 || _____ 10) {
 return -1; // Not a valid WQU
 }

 _____ 11;
 }

return _____ 12;

}

4 Asymptotics

(16 points)

For each of the following statements, select whether you think the statement is always **True**, the statement is sometimes **False**, or that you **Decline** to answer.

Within each set of five, the following rubric will be used to score your answer:

- Each correct answer will give **+2** points.
- Each incorrect answer will give **-3** points.
- Answers marked “Decline” or left blank will be awarded **0** points.
- The maximum and minimum scores for each set of five is **8** points and **0** points, respectively.
- Example: 4 correct + 1 “Decline”. You receive +8 total points, which gets capped to 8/8.
- Example: 2 correct + 3 incorrect. You receive -5 total points, which gets capped to 0/8.

Set I (8 points)

Let f and g be real-valued, **positive**, strictly increasing functions.

- If $f(x) \in O(g(x))$, then $f(10^{100}) \leq g(10^{100})$.
- If $f(x)$ and $g(x)$ are the best- and worst-case runtimes of a program, respectively, then $f(x) \in O(g(x))$.
- For all functions $f(x)$, there exists some function $g(x) \neq f(x)$ such that $f(x) \in \Theta(g(x))$.
- If $f(x) \in \Theta(g(x))$, then $f^{-1}(x) \in \Theta(g^{-1}(x))$, where $f^{-1}(x)$ is the inverse function of f . (The inverse function of f is the unique function f^{-1} such that if $y = f(x)$, then $x = f^{-1}(y)$.)
- If a program uses $O(\log(N))$ memory in total (including inputs, local variables, and recursive function calls), then its worst-case runtime is $O(N^k)$ for some constant k .

- | | | |
|--------------------------------|-------------------------------|-----------------------------|
| (a) ☐ True | ☐ Decline | ☐ False |
| (b) ☐ True | ☐ Decline | ☐ False |
| (c) ☐ True | ☐ Decline | ☐ False |
| (d) ☐ True | ☐ Decline | ☐ False |
| (e) ☐ True | ☐ Decline | ☐ False |

Set II (8 points)

- | | |
|--|---------------------------------|
| (a) $\sum_{i=1}^N i^2 \in \Theta(\sum_{i=1}^{N^2} i)$ | (d) $\log(N) \in O(N^{0.0001})$ |
| (b) $\sum_{i=1}^N \log_{10}(i) \in \Omega(\sum_{i=1}^N \log_2(i))$ | (e) $N! \in \Theta((N+1)!)$ |
| (c) $N^3 \in \Theta((N+1)^3)$ | |

- | | | |
|--------------------------------|-------------------------------|-----------------------------|
| (a) ☐ True | ☐ Decline | ☐ False |
| (b) ☐ True | ☐ Decline | ☐ False |
| (c) ☐ True | ☐ Decline | ☐ False |
| (d) ☐ True | ☐ Decline | ☐ False |
| (e) ☐ True | ☐ Decline | ☐ False |

5 Hashing

(12 points)

As mentioned in class, putting mutable objects in a hash table is dangerous, even if the `hashCode` does not change when an object is mutated. It turns out that no matter what clever hash code you try to devise, it is impossible to guarantee both efficiency and correctness when storing mutable objects. For each of the following `hashCode`s, describe a flaw/limitation in the proposed design. You should clearly state whether the issue is invalidity or inefficiency. If your answer is “invalidity”, give a sequence of operations that yields an incorrect result (either in code or clearly expressed in English). If your answer is “inefficiency”, briefly describe why. Answers with incorrect explanations will be given zero credit. Two Lists are considered equal if they have the same items in the same order.

- (a) (4 points) Use as a `hashCode` the memory address of the `List`, which doesn't change when the `List` gets mutated.

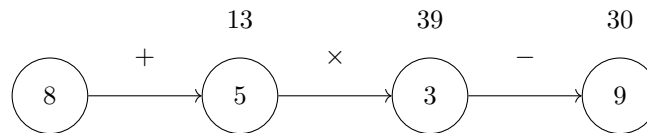
- (b) (4 points) Have `hashCode` always return 0.

- (c) (4 points) Create a new attribute `cachedHash` within the `List` class. When the `hashCode` method is first called on a `List`, compute the `hashCode` using `List<Integer>`'s normal `hashCode`, and save the result in `cachedHash`. After that, every future call of `hashCode` returns `cachedHash`, even if the list has been mutated.

6 Math Graphs

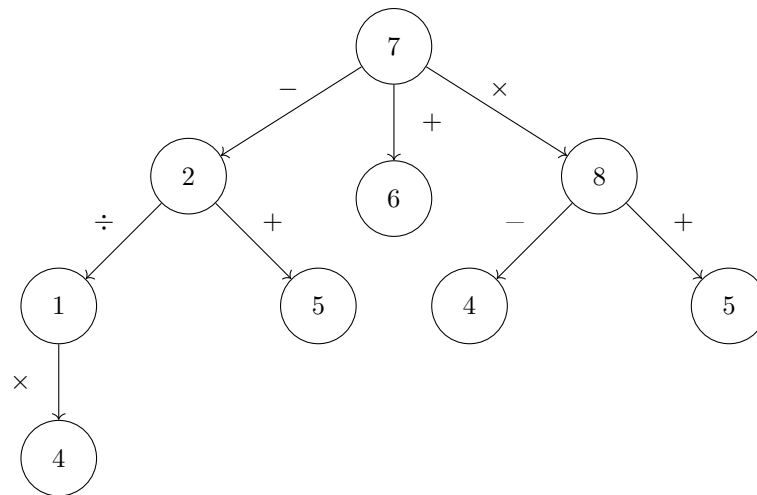
(11 points)

Aditya has created a type of graph to represent short arithmetic calculations called Math Graphs. In a Math Graph, each node contains a number, and each edge contains an arithmetic operation (addition, subtraction, multiplication, and floor division). A Math Path is a simple path in a Math Graph, and has a value equal to the result of computing the expression created along that path. For example:



The Math Path from 8 to 9 would have a value of $((8 + 5) \times 3) - 9 = 30$. Note that the value is computed in path order, and no operation has precedence over another.

(a) (1 point)



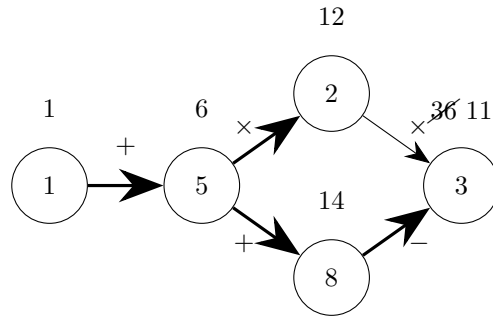
For the above Math Graph, what is the value of the largest-valued Math Path starting from 7? Write your answer as an integer.

(b) (1 point) True/False: If a Math Graph is a rooted tree, then its largest-valued Math Path will start at the root and end at a leaf.

☐ True

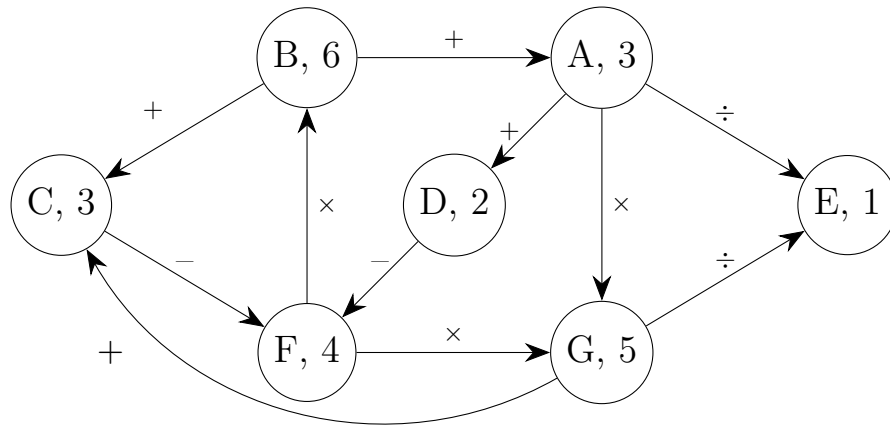
☐ False

Aditya wants to find the least-valued Math Path from one node to each other node of a Math Graph. He reasons that this is similar to finding the Shortest-Paths Tree of a graph, so he devises a slight modification to Dijkstra's algorithm. When evaluating a new path to a node, the appropriate math operation is performed between the current distance and the node's value instead of just adding an edge weight. For example:



When running Dijkstra's algorithm from node 1, we initially find a Math Path of value 36 to node 3. We later find a better path and relax it to a Math Path of value 11.

- (c) (6 points) Fill out the `distTo` and `edgeTo` arrays after running this modified version of Dijkstra's on the Math Graph below starting from node A:



Node	A	B	C	D	E	F	G
<code>distTo</code>	0						
<code>edgeTo</code>	-						

For each of the following types of Math Graph, determine if the algorithm above would **always** find a shortest paths tree (as defined previously). You may assume that all nodes are positive integers.

- (d) (1 point) A Math Graph with only addition (+) and multiplication ($\times$) operators.

☐ Always

☐ Not Always

- (e) (1 point) A Math Graph with NO division ($\div$) operators.

☐ Always

☐ Not Always

- (f) (1 point) A Math Graph that is a tree.

☐ Always

☐ Not Always